

SOFTWARE DESIGN DOCUMENT

Automated Web Data Acquisition and Preprocessing System for LLM Training

Final Design Version 1.0

Indian Institute of Information Technology, Pune

Authors

Abhay Gugale - MIS: 112415005

Aanoush Surana - MIS: 112415003

Date: 30 August 2026

Document Control

Field	Value
Document	Software Design Document (SDD)
Project	Automated Web Data Acquisition and Preprocessing System for LLM Training
Version	1.0
Date	30 August 2026
Status	Final design baseline
Methodology	Agile SDLC
Architecture	Layered Modular Architecture with Pipeline-Based Processing

Design Baseline and Scope Decisions

This document is the design baseline for the moderate-scale system. It converts the defined requirements and finalized design decisions into a concrete architecture, module structure, database model, interface design, processing flow, error-handling approach, and implementation structure. The design deliberately avoids production-scale distributed infrastructure while providing clear separation of concerns, testability, and extensibility.

- Backend: Python with FastAPI.
- Database: PostgreSQL.
- HTTP client: HTTPX.
- HTML parsing: BeautifulSoup with lxml.
- Content extraction: rule-based cleaning combined with readability/content extraction.
- URL normalization: standard normalization.
- Deduplication: URL-level deduplication; content hash generated as metadata, without content-level duplicate removal in Version 1.
- JavaScript: fallback rendering with Playwright only when normal retrieval is insufficient.
- PDFs: download plus extraction attempt with PyMuPDF; original PDF is retained if usable text cannot be extracted.
- Crawling: breadth-first search (BFS) with configurable depth.
- Retry: maximum three attempts with exponential backoff.
- Timeout: configurable, default 15 seconds.
- Rate limiting: per-domain pacing plus a concurrency limit.
- robots.txt: configurable behavior; default design recommendation is to respect it.
- Performance: measurable moderate targets validated through a benchmark.
- Output: structured JSON dataset plus media files and operational logs.
- Authentication and scheduled crawling are outside Version 1.

Database scope note: the preceding selections did not explicitly choose the earlier database-scope question. For the design to satisfy the stated job-status, URL-tracking, page-result, error, and media-metadata requirements, this SDD treats PostgreSQL as storing jobs, URLs, processed page records, errors, and media metadata, while the actual image/PDF files remain on the filesystem. This is a design interpretation of the selected PostgreSQL/storage choice rather than a new user-facing requirement.

Table of Contents

1. Introduction
 2. System Overview
 3. Architecture Design
 4. Module and Component Design
 5. Data and Database Design
 6. Detailed Processing Design
 7. User Interface Design
 8. API and Interface Design
 9. Error Handling and Logging
 10. Security and Safety Design
 11. Performance and Resource Design
 12. Deployment and Configuration
 13. Testing Design
 14. Requirement Traceability
 15. Design Constraints, Assumptions and Future Extensions
 16. Conclusion
- Appendix A - Data Dictionary
- Appendix B - Representative JSON Schema

1. Introduction

1.1 Purpose

The purpose of this design document is to define how the Automated Web Data Acquisition and Preprocessing System will be constructed. It specifies the system architecture, module responsibilities, data model, processing flow, interfaces, user interface, error handling, security controls, performance considerations, deployment model, and testing approach.

1.2 Design Objectives

- Maintain a clear separation between presentation, application, processing, and data responsibilities.
- Provide independent modules that can be developed and tested separately.
- Continue processing when individual pages fail.
- Support static webpages and basic JavaScript-rendered pages without rendering every page in a browser.
- Produce structured records suitable for downstream NLP/LLM data preparation.

2. System Overview

2.1 System Purpose and Scope

The system is a web application that accepts one or more HTTP/HTTPS seed URLs, executes a configurable crawl, processes retrieved documents, and produces structured output. Version 1 does not include user authentication, scheduled crawling, CAPTCHA bypass, authentication bypass, paywall bypass, or other access-control bypass functionality.

2.2 Actors

Actor	Responsibilities
Normal User	Provides seed URLs, configures jobs, starts or monitors jobs, views results, and exports JSON data.
Data Analyst / Researcher	Uses collected and preprocessed data for NLP/LLM-oriented datasets and analysis.
External Web Server	Provides HTTP/HTTPS resources requested by the crawler.
System Administrator / Developer	Maintains configuration, dependencies, database, logs, and deployment environment.

2.3 End-to-End Workflow

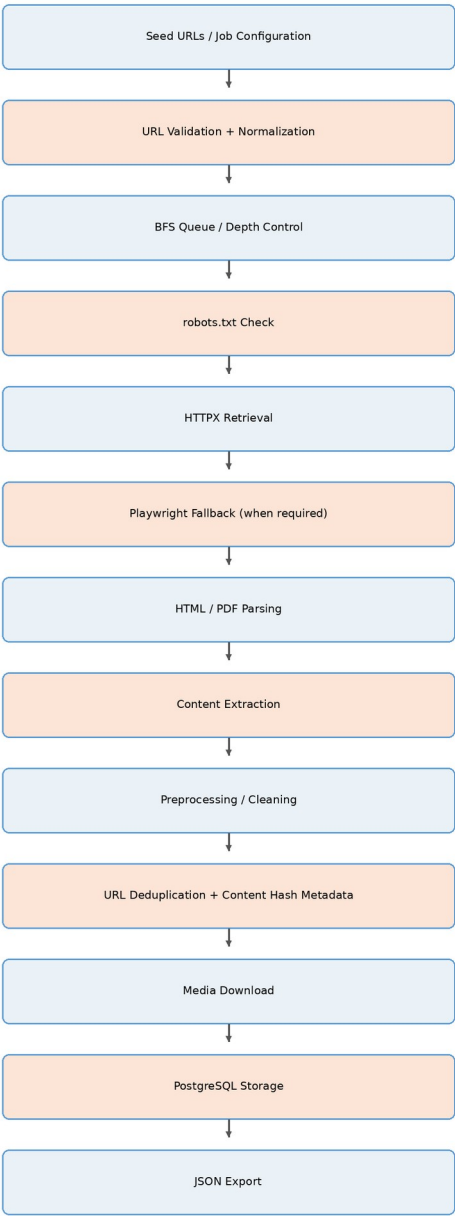


Figure 1. End-to-end processing pipeline.

2.4 Major Functional Areas

Area	Primary Function
Job Management	Create, configure, start, monitor, and complete scraping jobs.
URL Management	Validate, normalize, queue, track, and deduplicate URLs.
Crawling	Follow discovered links according to BFS and configured crawl depth.
Retrieval	Fetch content using HTTPX with timeouts, retries, pacing, and status handling.
Rendering	Use Playwright as a fallback for pages requiring basic JavaScript execution.
Parsing and Extraction	Parse HTML/PDF content and extract titles, headings, text, links, tables, and media references.

Area	Primary Function
Preprocessing	Remove noise and normalize extracted content.
Storage and Export	Persist metadata/results in PostgreSQL and produce JSON output plus media files.
Monitoring and Errors	Track status, failures, retry attempts, and job statistics.

3. Architecture Design

3.1 Architecture Selection

Three broad architecture alternatives were considered: a simple monolithic design, a layered modular monolith, and a distributed microservice design. The selected architecture is a layered modular design with a pipeline-based processing subsystem. It is appropriate for a moderate-scale system because it demonstrates separation of concerns and extensibility without introducing the operational complexity of independently deployed services.

Architecture	Strengths	Weaknesses	Decision
Simple Monolith	Lowest implementation complexity; easy deployment.	Weak separation of concerns; harder independent testing and extension.	Not selected
Layered Modular + Pipeline	Clear responsibilities; testable; understandable; good extensibility; manageable infrastructure.	Still a single application; less scalable than distributed systems.	SELECTED
Microservices	Independent deployment and scaling.	Requires service discovery, messaging, deployment, distributed debugging and more infrastructure than the project needs.	Rejected

3.2 Selected Architecture

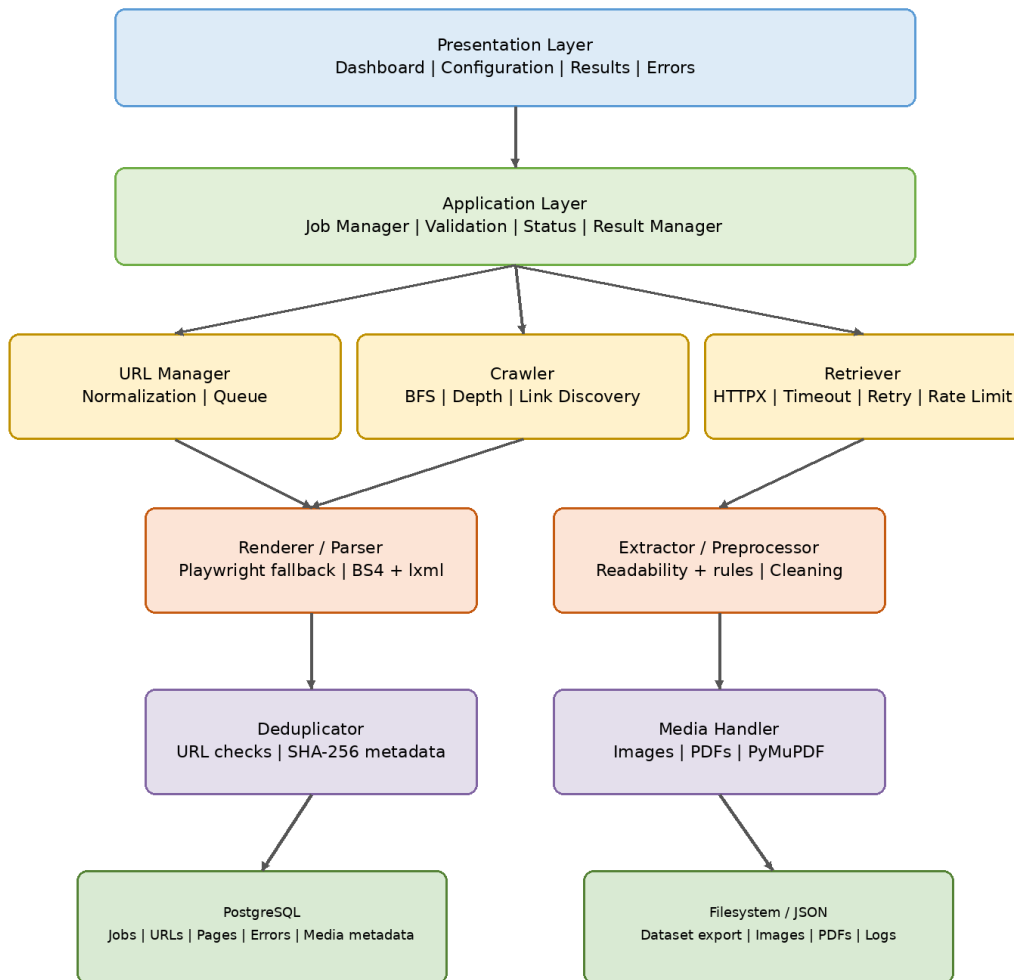


Figure 2. Selected layered modular architecture.

3.3 Architectural Layers

Layer	Responsibilities	Examples
Presentation	User interaction, forms, dashboards, result display.	FastAPI routes/templates, HTML/CSS/JS
Application	Coordinates jobs, validates configuration, manages state and results.	JobManager, ValidationManager, StatusManager
Processing	Performs crawling, retrieval, rendering, parsing, extraction, preprocessing, deduplication, and media handling.	Crawler, Retriever, Renderer, Parser, Extractor, Preprocessor
Data	Persistence and output.	PostgreSQL, JSON exporter, filesystem storage

3.4 Processing Pipeline Principles

- Each processing component consumes a defined input object and produces a defined output object.

- Modules communicate through application-level service interfaces rather than direct database manipulation wherever practical.
- The pipeline is fault-tolerant at the URL level: a failed URL becomes a recorded failure and the job continues.
- Resource-intensive browser rendering is only invoked when the static retrieval path is insufficient.
- Persistent state is written at meaningful checkpoints so job progress can be monitored.

4. Module and Component Design

4.1 FastAPI Web Layer

Exposes pages and REST-style endpoints for job configuration, status, results, errors, and export. It should not contain crawling logic.

4.2 Job Manager

Creates jobs, initializes configuration, starts processing, tracks counters, and transitions jobs through CREATED, RUNNING, COMPLETED, COMPLETED_WITH_ERRORS, or FAILED states.

4.3 Configuration Manager

Normalizes and validates job settings such as crawl depth, recursion, rendering mode, robots mode, timeout, retries, rate limit, and concurrency.

4.4 URL Manager

Normalizes URLs, creates queue entries, stores crawl depth, checks visited/processed state, and supplies the next BFS URL.

4.5 BFS Crawler

Processes the URL queue breadth-first, extracts links from successful documents, increments depth, and stops link expansion when the configured depth is reached.

4.6 HTTP Retriever

Uses HTTPX to retrieve resources over HTTP/HTTPS. It handles timeouts, HTTP status codes, request headers, retries, backoff, and interaction with rate/concurrency controls.

4.7 Rate Limiter

Applies per-domain request pacing and a configurable concurrency ceiling to prevent uncontrolled request bursts.

4.8 JavaScript Renderer

Uses Playwright for basic JavaScript-rendered pages when static retrieval is inadequate. It returns rendered HTML to the normal parsing path.

4.9 Document Parser

Parses HTML with BeautifulSoup/lxml and extracts text-bearing structures. PDF documents are handled through PyMuPDF.

4.10 Content Extractor

Identifies primary content using rules and readability-style extraction; collects title, headings, paragraphs, links, tables, and media references.

4.11 Preprocessor

Removes boilerplate/noise, normalizes whitespace and encoding, and produces clean text while preserving meaningful source content.

4.12 Deduplicator

Checks normalized URLs before processing and generates a content hash for each processed document. Content hash is metadata in Version 1, not a content-removal mechanism.

4.13 Media Downloader

Downloads encountered images and PDF files, stores local paths, records metadata, and attempts PDF text extraction.

4.14 Storage Manager

Writes job, URL, page, error and media metadata to PostgreSQL and coordinates JSON export and local media paths.

4.15 Error Manager

Categorizes errors, records timestamps/URL/job context and attempt counts, updates job statistics, and prevents page-level errors from unnecessarily terminating a job.

4.16 JSON Exporter

Builds a structured JSON dataset from processed records. The schema includes document metadata and extracted content structures.

4.17 Suggested Project Structure

```

project/
├── app.py
├── config/
│   ├── settings.py
│   └── database.py
├── routes/
│   ├── dashboard.py
│   ├── jobs.py
│   └── results.py
├── models/
│   ├── job.py
│   ├── url_record.py
│   ├── page.py
│   ├── media.py
│   └── error_log.py
├── services/
│   ├── job_manager.py
│   ├── url_manager.py
│   ├── crawler.py
│   ├── retriever.py
│   ├── renderer.py
│   ├── parser.py
│   ├── extractor.py
│   ├── preprocessor.py
│   ├── deduplicator.py
│   ├── media_downloader.py
│   ├── json_exporter.py
│   └── error_manager.py
├── utils/
│   ├── url_normalizer.py
│   ├── retry.py
│   ├── rate_limiter.py
│   ├── hashing.py
│   └── logging.py
├── templates/
├── static/
├── output/
└── tests/

```

5. Data and Database Design

5.1 Persistence Strategy

PostgreSQL is the system of record for job and processing metadata. JSON is the user-facing dataset/export format. Images and PDFs are kept as files rather than binary database objects, while their paths, source URLs, types, sizes, and processing status are stored in PostgreSQL.

5.2 Entity Relationship Model

```

ScrapingJob 1 ----- N URLRecord 1 ----- 0..1 Page
|               |               |
|               |               +----- N Media
|               |
+-----+-----+----- N ErrorLog

```

Key relationships:

- One job contains many URL records.
- A successfully processed URL may have one page record.
- A page may reference many downloaded media files.
- Errors may be associated with a job and, when applicable, a URL.

5.3 Database Tables

Table	Key Fields	Purpose
scraping_jobs	job_id PK, status, crawl_depth, recursive, counters, timestamps	Stores job configuration and overall execution status.
url_records	url_id PK, job_id FK, url, normalized_url, depth, status	Tracks every discovered/processed URL and crawl depth.
pages	page_id PK, url_id FK, title, content, content_hash, timestamp	Stores processed textual content and document metadata.
media	media_id PK, page_id FK, media_type, source_url, local_path, status	Stores metadata for images and PDF files.
error_logs	error_id PK, job_id FK, url_id FK nullable, error_type, message, attempt, timestamp	Stores categorized errors and retry context.

5.4 Important Database Constraints and Indexes

- Primary keys are generated uniquely for jobs, URLs, pages, media, and errors.
- Foreign keys preserve job-to-URL-to-page relationships.
- A unique index on normalized_url should prevent accidental duplicate URL records within the same job where appropriate.
- An index on url_records(job_id, status) supports progress and failure queries.
- An index on pages(content_hash) allows future content-level duplicate detection without requiring it in Version 1.
- Timestamps are stored for creation, discovery, processing, and errors.

5.5 Job State Model

CREATED -> RUNNING -> COMPLETED
 -> COMPLETED_WITH_ERRORS
 -> FAILED

A job enters FAILED only when a job-level failure prevents further execution.
 Individual page failures normally produce COMPLETED_WITH_ERRORS when the job
 otherwise finishes successfully.

6. Detailed Processing Design

6.1 Job Creation

1. User submits one or more seed URLs and job settings.
2. Validation manager checks URLs and configuration values.
3. Job record is inserted into PostgreSQL with CREATED status.
4. Seed URLs are normalized and inserted into the BFS queue.
5. Job status changes to RUNNING when execution begins.

6.2 URL Validation and Normalization

Only HTTP and HTTPS schemes are accepted. Standard normalization is applied before duplicate checks. The design removes fragments, normalizes scheme/host casing, handles default ports, and applies conservative path normalization. Query parameters are not removed indiscriminately because they may represent distinct resources.

6.3 BFS Crawling

Breadth-first traversal is used because crawl depth becomes explicit and a queue naturally represents the frontier of pages to process. Recursive crawling controls whether discovered links are added to the queue.

```
queue = [(seed_url, 0)]
visited = set()

while queue:
    url, depth = queue.pop(0)
    if normalized(url) in visited:
        continue
    visited.add(normalized(url))
    process(url, depth)
    if recursive and depth < max_depth:
        for link in discovered_links:
            queue.append((link, depth + 1))
```

6.4 robots.txt Behavior

robots.txt handling is configurable. The crawler can either respect robots.txt or operate without using it as a crawling restriction. The default configuration is Respect.

6.5 HTTP Retrieval, Timeout and Retry

HTTPX performs the normal retrieval path. The default timeout is 15 seconds and remains configurable. Up to three attempts are permitted for retryable failures, with exponential backoff. Permanent client-side errors and non-retryable HTTP responses are recorded without repeated retries.

6.6 JavaScript Rendering Fallback

A page is first retrieved through HTTPX. When the returned document appears incomplete for the configured extraction path or requires basic client-side rendering, Playwright may be invoked. Rendering is therefore an exception path, which reduces browser overhead.

6.7 HTML and PDF Parsing

HTML is parsed with BeautifulSoup using lxml as the parser. Titles, headings, paragraphs, links, tables, and media references are retained as structured extraction targets. PDFs are downloaded and passed to PyMuPDF for text extraction. When extraction fails or returns unusable text, the original PDF is retained and the failure is recorded rather than stopping the job.

6.8 Content Extraction and Preprocessing

- Remove script, style, navigation, and other obvious non-content elements where appropriate.
- Use readability/content heuristics to identify the primary content region.
- Preserve titles, headings, links, tables, image references, and relevant text.
- Normalize whitespace, line breaks, and encoding artifacts.
- Avoid aggressive NLP transformations such as stopword removal or stemming because the objective is to preserve useful source text for downstream LLM/NLP processing.

6.9 Deduplication

URL-level deduplication occurs before repeated processing. A SHA-256 content hash is generated after preprocessing and stored as metadata. Version 1 does not remove a document solely because its content hash matches another document; content-level duplicate removal is outside Version 1.

6.10 Media Processing

Image and PDF URLs encountered during extraction are queued for download under the same request pacing and error-handling rules. Files are stored under job-specific output folders and referenced from PostgreSQL by local path and source URL.

6.11 JSON Generation

After processing, the exporter creates one structured JSON record per processed document or a job-level JSON collection, depending on the implementation choice. The design supports titles, headings, paragraphs, combined content, tables, links, images, documents, and metadata such as timestamps, status, crawl depth, and content hash.

7. User Interface Design

7.1 UI Goals

- Simple configuration for a normal user.
- Visible job progress and statistics.
- Clear indication of successes and failures.
- Easy access to processed records and JSON export.
- No authentication workflow in Version 1.

7.2 Main Screens

Screen	Main Elements	Actions
Dashboard	Recent jobs, counts, statuses, links.	Create job, open job details, view results.
Create Job	Seed URLs, crawl depth, recursion, rendering, robots mode, timeout, retries, rate/concurrency.	Validate and start job.
Job Monitor	Progress bar, URL counters, current status, current URL, errors.	Refresh/inspect job and errors.
Results	Document list, title, URL, status, content summary, metadata.	Open record, inspect media, export JSON.
Errors	Categorized error list with URL, type, attempt, timestamp.	Inspect troubleshooting information.

7.3 Job Creation Wireframe

```

+-----+
| New Scraping Job |
+-----+
| Seed URLs |
| [ https://example.com ] |
| [ https://example.org ] |
| |
| Crawl depth: [ 2 ] Recursive: [x] |
| JS fallback: [x] Respect robots.txt: [x] |
| Timeout (sec): [15] Retries: [3] |
| Domain rate: [2 req/s] Concurrency: [3] |
| |
| [ Validate & Start ] |
+-----+

```

7.4 Job Monitor Wireframe

```

+-----+
| Job JOB-001           Status: RUNNING           |
+-----+
| Progress: [#####-----] 75%                  |
| Total URLs: 100   Processed: 75   Successful: 70   |
| Failed: 5         Discovered: 94                  |
| Current URL: https://example.com/page42          |
| [ View Results ] [ View Errors ] [ Refresh ]      |
+-----+

```

8. API and Interface Design

8.1 Application API

Method	Endpoint	Purpose
POST	/api/jobs	Create a scraping job.
GET	/api/jobs	List jobs.
GET	/api/jobs/{job_id}	Get job details and status.
POST	/api/jobs/{job_id}/start	Start a created job if execution is separated from creation.
GET	/api/jobs/{job_id}/results	Retrieve processed result records.
GET	/api/jobs/{job_id}/errors	Retrieve categorized errors.
GET	/api/jobs/{job_id}/export	Generate/retrieve JSON export.

8.2 Example Create-Job Request

```

{
  "seed_urls": ["https://example.com"],
  "crawl_depth": 2,
  "recursive": true,
  "javascript_fallback": true,
  "robots_mode": "respect",
  "timeout_seconds": 15,
  "max_retries": 3,
  "domain_rate_limit": 2,
  "max_concurrency": 3
}

```

8.3 Internal Interfaces

Component	Input	Output
Crawler	Normalized URL + crawl context	Discovered URL objects
Retriever	URL + request policy	Retrieved document / retrieval error
Parser	Retrieved document	Parsed document
Extractor	Parsed document	Structured extracted content
Preprocessor	Extracted content	Processed document
Deduplicator	URL + processed content	Dedup result + content hash
Storage Manager	Processed document + metadata	Persistent IDs/status
JSON Exporter	Stored page records	Structured JSON

9. Error Handling and Logging

9.1 Error Categories

Category	Examples	Default Response
URL_VALIDATION_ERROR	Malformed URL, unsupported scheme.	Do not queue; report to user.
NETWORK_ERROR	DNS/connection failure.	Retry if eligible; then mark URL failed.
TIMEOUT_ERROR	Request exceeds timeout.	Retry with backoff; then fail URL.
HTTP_ERROR	4xx/5xx status.	Classify; retry only when appropriate.
PARSING_ERROR	Malformed or unsupported document.	Record error; continue job.
EXTRACTION_ERROR	Primary content cannot be identified.	Record partial/failure result as appropriate.
JS_RENDERING_ERROR	Browser rendering fails.	Record error; continue job without terminating entire crawl.
PDF_EXTRACTION_ERROR	No usable PDF text.	Retain original PDF and log extraction failure.
MEDIA_DOWNLOAD_ERROR	Image/PDF download failure.	Record failure; continue page/job.
DATABASE_ERROR	Persistence failure.	Record as job-level failure when persistence cannot safely continue.

9.2 Logging Design

Operational logs should include timestamp, log level, component, job ID, URL when applicable, event type, attempt number, and a concise message. Sensitive values or internal stack traces should not be exposed through the user interface.

9.3 Retry Policy

Retryable failure:
 Attempt 1 -> wait ~1 s -> Attempt 2 -> wait ~2 s -> Attempt 3
 If still unsuccessful -> record final failure -> continue job

Non-retryable failure:
 Record immediately -> mark URL failed -> continue job

10. Security and Safety Design

10.1 Input Validation

- Accept only HTTP/HTTPS seed URLs.
- Reject malformed URLs and unsupported schemes.
- Validate crawl settings against sane numeric ranges.
- Normalize URLs before storage and duplicate comparison.

10.2 Access and Web Safety

- The system must not bypass CAPTCHA, authentication, paywalls, or other access-control mechanisms.
- Request pacing and concurrency limits reduce the risk of excessive request bursts.
- robots.txt behavior is configurable as finalized in this design; the default should be respect.
- Downloaded files should be stored under controlled application directories.

10.3 Security of Application Data

- Use parameterized database queries or an ORM to prevent SQL injection.
- Escape content before displaying it in the web UI to reduce cross-site scripting risk from crawled content.

- Avoid storing sensitive user data that is not required.
- Do not expose database credentials or filesystem internals through error messages.

11. Performance and Resource Design

11.1 Performance Targets

Metric	Design Target / Policy
Default HTTP timeout	15 seconds; configurable.
Maximum retries	3 attempts.
Concurrency	Configurable application-level limit.
Per-domain pacing	Configurable request rate, applied independently by domain.
Crawl depth	Configurable; no fixed architectural maximum beyond safe application limits.
Benchmark dataset	50-100 representative URLs for benchmarking.
Progress reporting	Persist URL and job status so UI can display current counters.
Failure isolation	A single page failure should not terminate the entire job.

11.2 Benchmark Plan

6. Prepare a fixed test dataset containing static pages, pages with links, pages containing tables/media, JavaScript-dependent examples, and PDFs.
7. Measure total job duration, successful/failed URL counts, average retrieval time, and browser-fallback count.
8. Repeat the benchmark after implementation changes to detect performance regressions.
9. Report observed results rather than claiming a throughput figure before measurement.

11.3 Resource Management

- Do not render every page with Playwright.
- Limit concurrent retrieval and browser contexts.
- Use bounded queues and configurable crawl limits.
- Store media on disk rather than in database blobs.
- Close HTTP responses and browser contexts promptly.

12. Deployment and Configuration

12.1 Deployment Model

The intended deployment is a single general-purpose computer or server. FastAPI, the processing modules, PostgreSQL, and the filesystem output area may reside on the same machine for Version 1.

12.2 Runtime Components

Component	Role
FastAPI application	Provides web UI and API.
Processing modules	Execute crawl and preprocessing pipeline.
PostgreSQL	Stores persistent metadata and status.
Playwright browser runtime	Optional browser execution for JavaScript fallback.
Filesystem	Stores JSON exports, images, PDFs and logs.

12.3 Configuration Example

```
APP_ENV=development
DATABASE_URL=postgresql://<user>:<password>@localhost/<database>
REQUEST_TIMEOUT=15
MAX_RETRIES=3
MAX_CONCURRENCY=3
DOMAIN_RATE_LIMIT=2
ROBOTS_MODE=respect
OUTPUT_DIR=./output
LOG_LEVEL=INFO
```

13. Testing Design

13.1 Testing Strategy

The modular architecture supports testing at multiple levels. Unit tests validate individual functions; integration tests validate communication between modules and PostgreSQL; system tests validate complete jobs from seed URL to JSON output; benchmark tests validate resource and timing behavior.

Test Level	Examples
Unit	URL normalization, hash generation, BFS queue behavior, retry calculation, preprocessing rules.
Integration	Retriever + parser, parser + extractor, storage + exporter, job manager + database.
System	Create a job, crawl a small site, process media, export JSON, inspect errors.
Negative	Invalid URL, timeout, malformed HTML, unavailable PDF, repeated URL, rendering failure.
Performance	50-100 URL benchmark, concurrency comparison, browser-fallback overhead.

13.2 Representative Test Cases

ID	Scenario	Expected Result
TC-01	Valid seed URL	Job created and URL queued.
TC-02	Invalid URL	Validation error shown; URL not crawled.
TC-03	Duplicate discovered URL	Normalized duplicate skipped.
TC-04	Crawl depth reached	New links beyond configured depth are not queued.
TC-05	HTTP timeout	Retry policy runs; URL eventually marked failed if all attempts fail.
TC-06	JavaScript page	Playwright fallback retrieves renderable HTML.
TC-07	PDF with text	PDF saved and text extracted/preprocessed.
TC-08	PDF without usable text	PDF retained; extraction error recorded; job continues.
TC-09	Page parsing failure	URL marked failed; other queued URLs continue.
TC-10	Content hash generated	SHA-256 stored in page metadata without deleting matching content.

14. Requirement Traceability

The following mapping shows how the major functional requirements are realized by the selected design. Requirements REQ-F-001 through REQ-F-026 cover job configuration, retrieval, parsing, extraction, preprocessing, URL deduplication, storage/export, and error handling.

Requirement Group	Design Realization
REQ-F-001 to REQ-F-005	FastAPI job-creation interface + Job Manager + Configuration Manager.
REQ-F-006 to REQ-F-011	HTTPX Retriever + timeout/retry/rate limiting + Parser + Playwright fallback.
REQ-F-012 to REQ-F-016	Content Extractor + Preprocessor + Media Downloader + document parsers.
REQ-F-017 to REQ-F-019	URL Manager + standard URL normalization + PostgreSQL URL records.
REQ-F-020 to REQ-F-022	JSON Exporter + PostgreSQL Page metadata + structured JSON schema.
REQ-F-023 to REQ-F-026	Error Manager + retry policy + URL status + job continuation.

14.1 Nonfunctional Requirement Mapping

Design Area	Design Response
Performance	Configurable timeout, pacing, concurrency, benchmark plan.
Safety	Request control; no access-control bypass; configurable robots policy.
Security	URL validation, safe file handling, escaped UI output, limited error disclosure.
Reliability	URL-level failure isolation and explicit job states.
Maintainability	Layered modular architecture and separate services.
Usability	Dashboard, configuration screen, monitor, results and errors.
Testability	Independent components, unit/integration/system test layers.
Extensibility	Interfaces designed so scheduling, new parsers, and additional export formats can be added later.

15. Design Constraints, Assumptions and Future Extensions

15.1 Constraints

- Extraction quality depends on webpage structure and content layout.
- JavaScript-heavy pages may still be partially unsupported despite the fallback renderer.
- The system does not bypass access restrictions.
- Network availability and target-server behavior affect crawl performance.
- Media downloads consume disk space.
- Content-level deduplication is not implemented as a Version 1 filtering rule.

15.2 Assumptions

- The user provides accessible HTTP/HTTPS URLs.
- The host machine has Internet access and sufficient disk space.
- PostgreSQL is available to the application.
- The target webpages do not require prohibited bypass mechanisms.
- The benchmark dataset is representative of intended usage.

15.3 Future Extensions

- Scheduled crawling.
- Authentication and multi-user access control.
- Content-level deduplication using stored hashes.
- More advanced document formats and OCR for image-only PDFs.
- Distributed workers if data volume grows substantially.
- Additional export formats such as CSV or Parquet.
- Advanced crawl prioritization and domain policies.

16. Conclusion

The finalized design provides a balanced architecture: more structured than a basic crawler, but intentionally below the complexity of a production-scale distributed platform. FastAPI provides the application layer, PostgreSQL provides persistent state, and the processing subsystem is organized as modular stages for URL management, crawling, retrieval, rendering, parsing, extraction, preprocessing, deduplication, and storage. The design provides clear implementation and testing boundaries for the system.

Appendix A - Data Dictionary

Entity / Field	Type (conceptual)	Description
scraping_jobs.job_id	UUID / string	Unique job identifier.
scraping_jobs.status	enum	CREATED, RUNNING, COMPLETED, COMPLETED_WITH_ERRORS, FAILED.
scraping_jobs.crawl_depth	integer	Maximum allowed link-following depth.
scraping_jobs.recursive	boolean	Whether discovered links are added for further crawling.
url_records.url	text	Original discovered URL.
url_records.normalized_url	text	Normalized URL used for duplicate comparison.
url_records.depth	integer	Crawl depth at which URL was discovered.
url_records.status	enum	QUEUED, PROCESSING, SUCCESS, FAILED, SKIPPED.
pages.title	text	Extracted document title.
pages.content	text	Final combined processed text.
pages.content_hash	text	SHA-256 hash of processed content.
media.media_type	enum/text	Image or PDF.
media.local_path	text	Filesystem path of downloaded media.
error_logs.error_type	enum/text	Categorized processing error.
error_logs.attempt	integer	Attempt number associated with error.

Appendix B - Representative JSON Schema

```
{
  "job_id": "JOB-001",
  "url": "https://example.com/article",
  "title": "Example Article",
  "headings": ["Introduction", "Results"],
  "paragraphs": ["Processed paragraph one..."],
  "content": "Combined processed text...",
  "tables": [],
  "links": [
    {"url": "https://example.com/page2", "text": "Related page"}
  ],
  "images": [
    {"url": "https://example.com/image.jpg", "local_path": "images/001.jpg"}
  ],
  "documents": [
    {"url": "https://example.com/file.pdf", "local_path": "pdfs/001.pdf"}
  ],
  "metadata": {
    "timestamp": "2026-08-30T18:30:00",
    "crawl_depth": 1,
    "content_hash": "<sha256>",
    "status": "success"
  }
}
```